

國立臺灣大學電機資訊學院電機工程學研究所



碩士論文

Graduate Institute of Electrical Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

基於大型語言模型之暫存器傳輸級電路生成：
代理式 IP 重用規劃與串接路由

Agentic IP-Reuse Planning and Cascade Routing
for LLM-Based RTL Generation

(待填姓名)

TODO-YOUR-NAME

指導教授：(待填指導教授) 博士

Advisor: TODO-ADVISOR-NAME, Ph.D.

中華民國 115 年 7 月

July 2026

國立臺灣大學碩士學位論文

口試委員會審定書



基於大型語言模型之暫存器傳輸級電路生成：
代理式 IP 重用規劃與串接路由

Agentic IP-Reuse Planning and Cascade Routing
for LLM-Based RTL Generation

本論文係（待填姓名）君（R00000000）在國立臺灣大學電機工程學研究所完成之碩士學位論文，於民國 115 年 7 月 31 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____	_____
_____	_____
_____	_____
_____	_____

所 長：_____





Acknowledgements

常到外國朋友家吃飯。當蠟燭燃起，菜肴布好，客主就位，總是主人家的小男孩或小女孩舉起小手，低頭感謝上天的賜予，並歡迎客人的到來。

我剛到美國時，常鬧得尷尬。因為在國內養成的習慣，還沒有坐好，就開動了。

以後凡到朋友家吃飯時，總是先囑咐自己；今天不要忘了，可別太快開動啊！幾年來，我已變得很習慣了。但我一直認為只是一種不同的風俗儀式，在我這方面看來，忘或不忘，也沒有太大的關係。

前年有一次，我又是到一家去吃飯。而這次卻是由主人家的祖母謝飯。她雪白的頭髮，顫抖的聲音，在搖曳的燭光下，使我想起兒時的祖母。那天晚上，我忽然覺得我平靜如水的情感翻起滔天巨浪來。

在小時候，每當冬夜，我們一大家人圍著個大圓桌吃飯。我總是坐在祖母身旁。祖母總是摸著我的頭說：「老天爺賞我們家飽飯吃，記住，飯碗裡一粒米都不許剩，要是糟蹋糧食，老天爺就不給咱們飯了。」

剛上小學的我，正在念打倒偶像及破除迷信等為內容的課文，我的學校就是從前的關帝廟，我的書桌就是供桌，我曾給周倉畫上眼鏡，給關平戴上鬍子，祖母的話，老天爺也者，我覺得是既多餘，又落伍的。



不過，我卻很尊敬我的祖父母，因為這飯確實是他們掙的，這家確實是他們立的。我感謝面前的祖父母，不必感謝渺茫的老天爺。

這種想法並未因為年紀長大而有任何改變。多少年，就在這種哲學中過去了。

我在這個外國家庭晚飯後，由於這位外國老太太，我想起我的兒時，由於我的兒時，我想起一串很奇怪的現象。

祖父每年在「風裡雨裡的咬牙」，祖母每年在「茶裡飯裡的自苦」，他們明明知道要滴下眉毛上的汗珠，才能撿起田中的麥穗，而為什麼要謝天？我明明是個小孩子，混吃混玩，而我為什麼卻不感謝老天爺？

這種奇怪的心理狀態，一直是我心中的一個謎。

一直到前年，我在普林斯頓，瀏覽愛因斯坦的我所看見的世界得到了新的領悟。

這是一本非科學性的文集，專載些愛因斯坦在紀念會上啦，在歡迎會上啦，在朋友的喪禮中，他所發表的談話。

我在讀這本書時忽然發現愛因斯坦想盡量給聽眾一個印象：即他的貢獻不是源於甲，就是由於乙，而與愛因斯坦本人不太相干似的。

就連那篇亙古以來嶄新獨創的狹義相對論，並無參考可引，卻在最後天外飛來一筆，「感謝同事朋友貝索的時相討論。」

其他的文章，比如奮鬥苦思了十幾年的廣義相對論，數學部份推給了昔年好友的合作：這種謙抑，這種不居功，科學史中是少見的。

我就想，如此大功而竟不居，為什麼？像愛因斯坦之於相對論，像我祖母之

於我家。

幾年來自己的奔波，做了一些研究，寫了幾篇學術文章，真正做了一些小貢獻以後，才有了一種新的覺悟：即是無論什麼事，得之於人者太多，出之於己者太少。因為需要感謝的人太多了，就感謝天罷。無論什麼事，不是需要先人的遺愛與遺產，即是需要眾人的支持與合作，還要等候機會的到來。越是真正做過一點事，越是感覺自己的貢獻之渺小。

於是，創業的人，都會自然而然的想到上天，而敗家的人卻無時不想到自己。





[illegible]

關鍵字：大型語言模型、暫存器傳輸級設計、IP 重用、代理式規劃、模型路由、硬體描述語言





Abstract

Abstract

Keywords: Large Language Models, RTL Generation, IP Reuse, Agentic Planning, LLM
Routing, Verilog

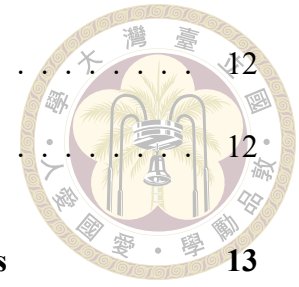




Contents

	Page
Verification Letter from the Oral Examination Committee	i
Acknowledgements	iii
摘要	vii
Abstract	ix
Contents	xi
List of Figures	xv
List of Tables	xvii
Denotation	xix
Chapter 1 Introduction	1
Chapter 2 Background and Related Work	3
Chapter 3 A Two-Stage Agentic IP-Reuse Pipeline	5
3.1 Overview and Environmental Contracts	5
3.2 Stage One: The Grounded Planner	7
3.3 Stage Two: The Generator	9
3.4 Repair Mechanisms	10
3.4.1 Syntax Repair Loop and the Semantic Repair Cache	10
3.4.2 Functional Repair Loop	11

3.4.3	Diagnostic-Driven Specification Slicing	12
3.5	Summary	12
Chapter 4	Cascade Routing between Direct and Pipeline Flows	13
4.1	Motivation: Two Complementary Flows	13
4.2	Task Classes and Oracle Labels	14
4.3	The Two-Tier Cascade	15
4.4	Routing Metrics	17
Chapter 5	Experimental Setup	19
5.1	The RealBench Benchmark	19
5.2	Models and Serving Infrastructure	21
5.3	Metrics and Cost Accounting	22
5.4	Run Inventory	23
Chapter 6	Results and Analysis	25
6.1	Does Planning and Reuse Beat the Model Alone?	25
6.2	The Grounding Audit, and an Honest Null Result	26
6.3	Sampling Structure: pass@1 to pass@20	27
6.4	Repair-Cache Ablation: Density, Not Solvability	28
6.5	Functional Repair: a Separate, Oracle-Fed Arm	29
6.6	Repair-Budget Calibration: Fix Fast or Never	30
6.7	Routing: Union Coverage at a Fraction of the Compute	31
6.8	Failure Taxonomy: Three Tiers of Difficulty	34
6.9	When the Benchmark Is Wrong: Scoring Artifacts	37



Chapter 7 Discussion, Limitations, and Conclusion

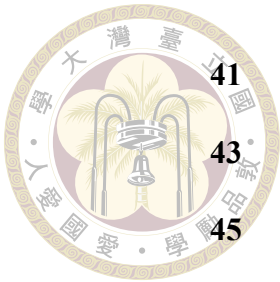
References

Appendix A — Introduction

A.1	Introduction	45
A.2	Further Introduction	45

Appendix B — Introduction 47

B.1	Introduction	47
B.2	Further Introduction	47







List of Figures

Figure 3.1	Two-stage pipeline dataflow	6
Figure 3.2	Semantic repair cache	11
Figure 4.1	Cascade router decision flow	15
Figure 6.1	Headline rates by task family	26
Figure 6.2	Grounding audit of reuse decisions	26
Figure 6.3	pass@k curves	28
Figure 6.4	Cache ablation metrics	29
Figure 6.5	Repair-attempt distributions	30
Figure 6.6	Routing frontier	32
Figure 6.7	Router confusion vs. oracle labels	33
Figure 6.8	Outcome taxonomy across runs	34
Figure 6.9	Verilator error classes	35
Figure 6.10	Per-module pass fraction across runs	36





List of Tables

Table 5.1	RealBench module-level task families	20
Table 5.2	Inventory of experimental runs	24
Table 5.3	Run labels to repository directories	24
Table 6.1	Headline comparison at temperature 0	25
Table 6.2	Repair-cache ablation	28
Table 6.3	Routing ablation arms	31





Denotation

HPC	高性能計算 (High Performance Computing)
cluster	集群
Itanium	安騰
SMP	對稱多處理
API	應用程序編程接口
PI	聚酰亞胺
MPI	聚酰亞胺模型化合物，N-苯基鄰苯酰亞胺
PBI	聚苯並咪唑
MPBI	聚苯並咪唑模型化合物，N-苯基苯並咪唑
PY	聚吡嚨
PMDA-BDA	均苯四酸二酐與聯苯四胺合成的聚吡嚨薄膜
ΔG	活化自由能 (Activation Free Energy)

χ 傳輸系數 (Transmission Coefficient)

E 能量

m 質量

c 光速

P 概率

T 時間

v 速度



勸學

君子曰：學不可以已。青，取之於藍，而青於藍；冰，水為之，而寒於水。木直中繩。輅以為輪，其曲中規。雖有槁暴，不覆挺者，輅使之然也。故木受繩則直，金就礪則利，君子博學而日參省乎己，則知明而行無過矣。吾嘗終日而思矣，不如須臾之所學也；吾嘗跂而望矣，不如登高之博見也。登高而招，臂非加長也，而見者遠；順風而呼，聲非加疾也，而聞者彰。假輿馬者，非利足也，而致千裏；假舟楫者，非能水也，而絕江河，君子生非異也，善假於物也。積土成山，風雨興焉；積水成淵，蛟龍生焉；積善成德，而神明自得，聖心備焉。故不積跬步，無以至千裏；不積小流，無以成江海。騏驥一躍，不能十步；駑馬十駕，功在不捨。鍤而舍之，朽木不折；鍤而不捨，金石可鏤。蚓無爪牙之利，筋骨之強，上食埃土，下飲黃泉，用心一也。蟹六跪而二螯，非蛇鱈之穴無可寄托者，用心躁也。——荀況



Chapter 1 Introduction





Chapter 2 Background and Related Work





Chapter 3 A Two-Stage Agentic IP-Reuse Pipeline

This chapter presents the first contribution of the thesis: a two-stage agentic pipeline that generates RTL by first planning which existing IP blocks to reuse and how to integrate them, and only then generating Verilog under a verify-and-repair loop. Section 3.1 gives the dataflow and the environmental contracts the pipeline must honor. Sections 3.2 and 3.3 describe the two stages. Section 3.4 describes the three repair mechanisms layered on the generator — the syntax repair loop with a semantic repair cache, the functional repair loop, and diagnostic-driven specification slicing. Design decisions are motivated here; their empirical evaluation is deferred entirely to Chapter 6.

3.1 Overview and Environmental Contracts

The pipeline processes one benchmark task end-to-end as follows (Figure 3.1):

1. **Catalog construction.** The task’s own dependency sources — and only those; no cross-task material ever enters, so there is no benchmark contamination — are indexed into a per-task reusable-IP catalog: one entry per provided module, carrying its name, port list, and source.

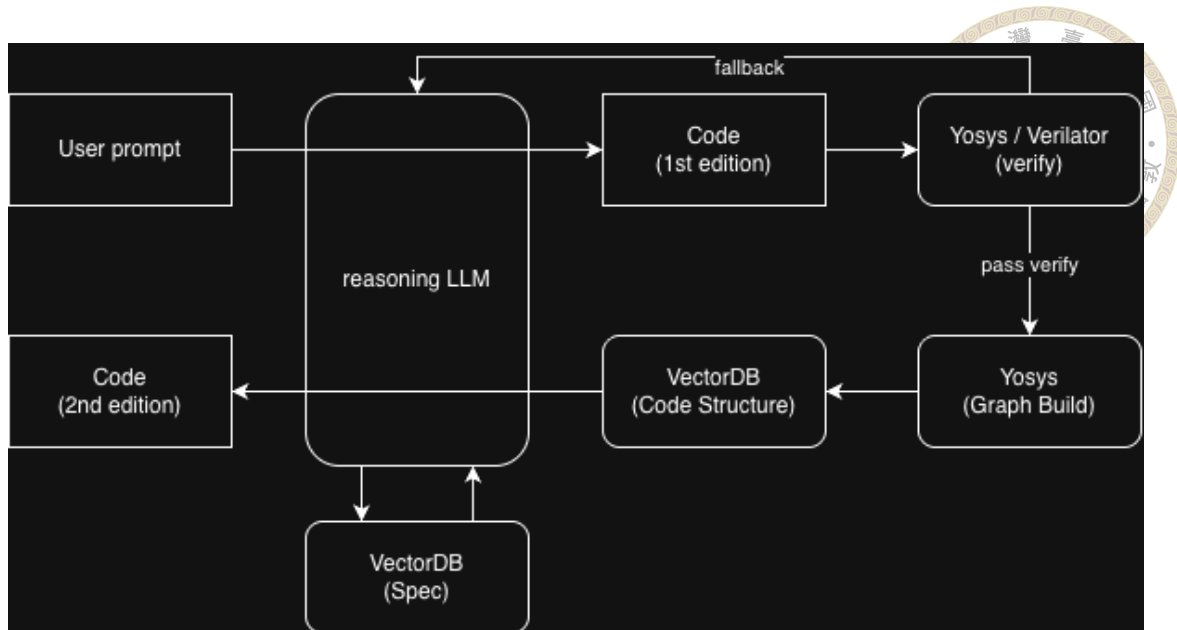


Figure 3.1: Dataflow of the two-stage pipeline. A RealBench task is expanded into a per-task IP catalog; the planner emits a grounded reuse plan; the plan adapter enriches it with source code and interface contracts; the generator produces Verilog inside a Verilator verify-and-repair loop before the final benchmark evaluation.

2. **Planning.** The planner (Section 3.2) reads the specification and the catalog and emits a structured IP-reuse plan: requirements, module decomposition, a reuse decision per submodule (reuse a catalog entry vs. write new RTL), and integration notes.
3. **Plan adaptation.** An adapter normalizes the plan, resolves each reuse decision to concrete source code, and assembles the generation prompt: the original specification, the signatures of reused modules, environment notes, and the testbench’s DUT instantiation verbatim as the binding interface contract.
4. **Generation and repair.** The generator (Section 3.3) produces the target module and iterates under an internal Verilator lint that compiles the generated code against the same file set the benchmark will use, feeding diagnostics back as repair prompts (Section 3.4).
5. **Evaluation.** The final candidate is scored by the untouched RealBench Makefile

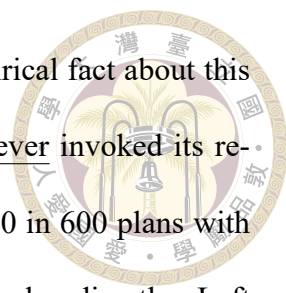
as described in Section 5.1.



The two fatal contracts. Two properties of the evaluation environment shape nearly every design decision downstream, and both are fatal when violated. First, the environment compiles every Verilog file in the task directory together, so any provided dependency module that the generated code re-declares produces a duplicate-module error (MODDUP): shipped dependencies must be instantiated, never re-implemented. (The single family-wide exception, `e203_extend_csr`, is not shipped as a file and must be inlined — the kind of irregularity that motivates grounding plans in the real catalog rather than in the model’s memory.) Second, the testbench instantiates the device under test with an exact port list; a generated module that misses one port fails with `PINNOTFOUND` regardless of how correct its internals are. The pipeline addresses the first contract by stripping re-declared modules and cataloging what is provided, and the second by injecting the testbench’s DUT instantiation into every generation and repair prompt as a hard interface contract.

3.2 Stage One: The Grounded Planner

The planner is a tool-calling agent equipped with a `search_reuse_ip` tool over the per-task catalog (token-overlap, semantic-embedding, or hybrid retrieval backends). Given the specification, it must return a structured JSON plan with a fixed schema: requirement summary, module decomposition, per-module reuse decisions (`selected_ip` naming a catalog entry, or `new_rtl_required`), and integration, verification, and debugging plans.



The models do not call tools. The single most consequential empirical fact about this stage is negative: across every configuration we ran, the planner never invoked its retrieval tool — 0 tool calls in 1,200 plans with gpt-oss-20B, and still 0 in 600 plans with the $6\times$ larger gpt-oss-120B. The models simply emit the structured plan directly. Left uncorrected, the reuse decisions in those plans are hallucinated from pretraining memory: plausible-but-wrong module names such as e203_exu_core where the real catalog has e203_exu. Rather than fighting the refusal (prompt pressure, forced tool choice, and re-prompting all failed to change it), the final design removes the dependency on tool use with three mechanisms that together we call grounding-by-construction:

1. **Catalog injection.** The real catalog vocabulary — every reusable module id with a one-line digest — is injected directly into the planner’s prompt, and the system prompt mandates a closed vocabulary: selected_ip must be one of the listed ids.
2. **Closed-vocabulary grounding.** Every selected_ip in the returned plan is post-hoc resolved against the catalog: exact matches kept; near misses remapped by prefix/substring matching and then fuzzy string similarity (cutoff 0.6); irrecoverable names dropped and the submodule marked new_rtl_required. The plan additionally records the grounding outcome per decision, making plans auditable.
3. **Completeness gate.** A task whose catalog is non-empty but whose plan contains no reuse and no integration content is re-prompted once (tool-free), and the more complete of the two attempts is kept.

Chapter 6 (Section 6.2) audits the effect: grounding eliminates hallucinated names from plans entirely, and — an equally important honest finding — barely moves the end-to-end pass rate, because the generator was already neutralizing bad names downstream.

Plans become correct and auditable; the binding constraint lies elsewhere.



Large specifications. Specifications beyond a token threshold (45,000 estimated tokens) are condensed before planning by a chunk-and-merge summarizer that renders two views from one manifest: a planner view (interfaces, reuse queries, and a brief behavioral digest) and a generation view (full requirements for to-be-generated logic; interface-only for provided IP). Port tables and module headers are carried verbatim from the raw specification past the LLM summarizer, because interface fidelity is exactly what the two fatal contracts punish. Condensation compresses the largest specifications by roughly $5\times$ (e.g., $\sim 720\text{ k} \rightarrow \sim 146\text{ k}$ characters) while keeping every shared fact and module interface stated exactly once.

3.3 Stage Two: The Generator

The generator receives the adapted plan and produces the target module with a code-focused prompt assembled from: the (possibly condensed) specification; the source and signatures of every reused module; the environment notes (which modules are provided as files, which macros the environment defines); and the testbench DUT instantiation as the interface contract. Generated text is parsed for a Verilog code block; provided dependency modules that the model re-declared anyway are stripped before verification.

Internal verification mirrors the benchmark. The generator’s inner loop lints each candidate with Verilator against the same file set the benchmark compiles — dependencies, reference model, and testbench, with the task Makefile’s warning waivers applied — rather than linting the generated file in isolation. This design decision followed a concrete

failure: an earlier isolated lint could not see testbench-side pin connections, so interface violations (PINNOTFOUND) sailed through internal checks and died only at evaluation. Mirroring the benchmark's compile closes that gap; on the runs where both verdicts exist for identical code, the internal lint agrees with the benchmark's syntax verdict on 60 of 60 tasks. The cost of this fidelity is that Verilator diagnostics can quote testbench or reference lines into repair prompts; we return to this leak surface in Section ??.

3.4 Repair Mechanisms

Three mechanisms sit on top of the generator's inner loop. Throughout the thesis, a repair budget " s/f " means at most s syntax-repair and f functional-repair attempts per sample.

3.4.1 Syntax Repair Loop and the Semantic Repair Cache

When the internal lint fails, the diagnostics are fed back to the model in a repair prompt together with the current candidate, and the process repeats up to the syntax budget. The loop is enhanced by a semantic repair cache (Figure 3.2) that reuses the model's own past successes: whenever a repair subsequently passes verification, the pair (diagnostic signature $\rightarrow$ unified-diff excerpt of the fix) is stored. Signatures are Verilator diagnostic lines with file paths and identifiers stripped, gated by error code, so that structurally identical errors on different modules collide. On a later failure, sufficiently similar cached entries are injected into the repair prompt — advisory only, never auto-applied, and never sourced from benchmark reference code. The cache has three scopes: off, task (shared only across one task's samples), and run (shared across the whole run). Run

scope maximizes hint availability but couples a task's samples, which corrupts pass@k's independence assumption — the trade-off is measured in Section 6.4.

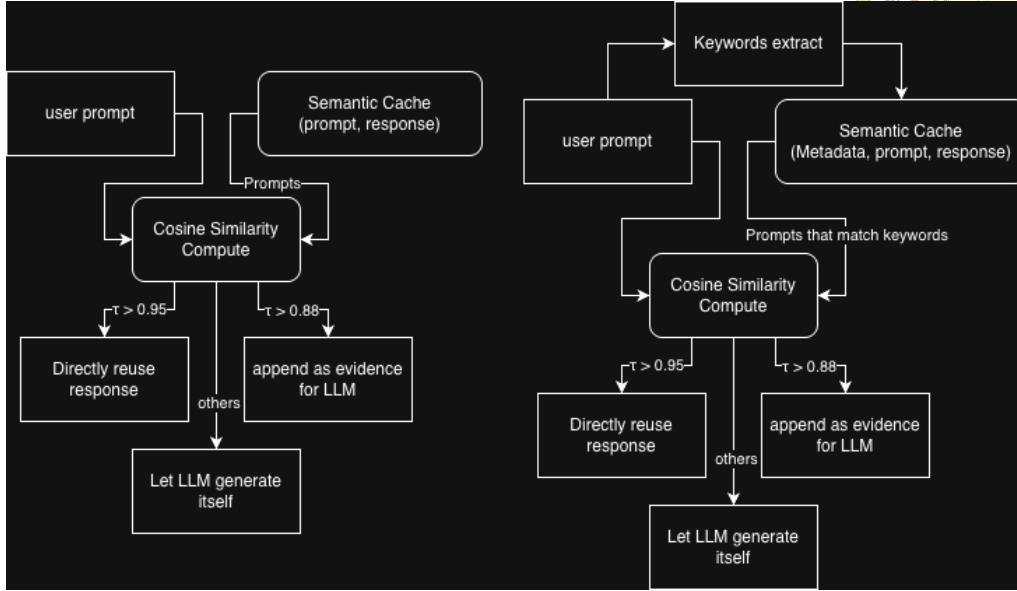


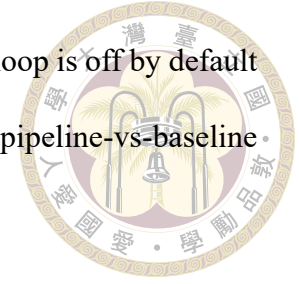
Figure 3.2: The semantic repair cache. Verified diagnostic→fix pairs from the model's own repairs are keyed by identifier-stripped diagnostic signatures and replayed as advisory hints when similar failures recur.

3.4.2 Functional Repair Loop

The syntax loop is structurally blind to the largest failure class: candidates that compile but fail the testbench (Section 6.8). The functional repair loop attacks this class directly. After a candidate passes syntax, the testbench is run; on mismatch, the simulator's mismatch report (“Output x has N mismatches, first at time T ”) is fed back in a prompt that frames the problem explicitly as logic repair — the interface is correct, do not change it — and the loop keeps the best compiling candidate by lowest mismatch count, never advancing to a candidate that regresses syntax.

One methodological property makes this loop different in kind from syntax repair, and we flag it prominently: the feedback signal comes from the same testbench that scores the benchmark. A pass obtained through functional repair therefore means “specification

plus oracle feedback,” not “specification alone.” For that reason the loop is off by default and reported as a separate experimental arm (Section 6.5); headline pipeline-vs-baseline comparisons never include it.



3.4.3 Diagnostic-Driven Specification Slicing

Repair prompts initially carried the full specification, which for large tasks buries the relevant sentence under tens of thousands of irrelevant tokens. Specification slicing exploits the fact that Verilator diagnostics and testbench mismatch reports name the offending identifiers exactly. The slicer extracts those identifiers, then assembles a focused slice: the verbatim interface excerpts (port tables, module headers) plus only the specification sections that mention an offending identifier, under a character budget. When a diagnostic names no identifier (e.g., a bare parse error), the slicer returns nothing and the full specification is kept, so repair is never starved of context. On the largest specification the slice is a $\sim 29\times$ reduction ($\sim 704\text{ k} \rightarrow 24\text{ k}$ characters).

3.5 Summary

The pipeline’s design distills to three commitments. Ground, don’t trust: every reuse decision is resolved against the real catalog because models plan from memory even when handed a retrieval tool. Verify against the real environment: the inner lint compiles exactly what the benchmark will compile, making the two fatal contracts visible during repair rather than at scoring. Reuse the model’s own verified successes: repair hints come from diffs that already fixed a structurally identical failure. What these commitments buy — and what they conspicuously fail to buy — is quantified in Chapter 6.



Chapter 4 Cascade Routing between Direct and Pipeline Flows

The pipeline of Chapter 3 is not uniformly better than simply asking the model for the module. On a substantial class of tasks it is worse: its planning and reuse scaffolding invents structure that a self-contained module does not need, and derails a generation that a bare prompt would have gotten right. This chapter presents the second contribution of the thesis: a per-task cascade router that predicts, before spending pipeline-scale compute, which of two flows a task should take — the full agentic pipeline, or a cheap direct single-shot generation. Section 4.1 establishes why the two flows are complementary; Section 4.2 defines the task classes and the oracle labels used to evaluate routing; Section 4.3 describes the two-tier cascade; and Section 4.4 defines the routing-specific metrics used in Section 6.7.

4.1 Motivation: Two Complementary Flows

Running both flows on all 60 tasks (the HYBRID arm of Table 5.2) reveals a clean division of labor. With ten samples per flow, the pipeline alone solves 20 tasks and the direct flow alone solves 22 — but their union solves 30, because the two sets overlap only

partially. Wrapper and integration modules pass under the pipeline, which supplies the exact interfaces of the submodules they must instantiate and wire; self-contained algorithmic modules pass under direct generation, which lets the model write the datapath in one coherent piece without the pipeline's decomposition machinery second-guessing it. Neither flow dominates: each one's strength is the other's failure mode.

The obvious exploitation of complementarity — run both flows on every task — is also the most expensive one, and it wastes most of its budget: every direct sample spent on a wrapper module and every pipeline run spent on a self-contained ALU is compute that predictably converts to nothing. A router that sends each task to its matching flow can, in principle, reach the union coverage at roughly half the cost. The routing problem is to make that prediction before the expensive flow has been paid for.

4.2 Task Classes and Oracle Labels

We formalize the division as two task classes:

Class A (structural). Wrapper, integration, and memory-encapsulation modules: the module's own logic is thin; its difficulty is wiring externally defined submodule interfaces correctly. Class A tasks want the **pipeline**.

Class B (algorithmic). Self-contained datapath or control modules — cipher rounds, CRC, instruction decode, address generation: the difficulty is the logic itself, not integration. Class B tasks want **direct** generation.

For evaluation only, each task receives an oracle label computed from its golden implementation: the reference RTL is synthesized with Yosys, submodule instances are

subtracted, and the task is labeled A if its own synthesized logic falls below a 250-cell threshold (or it is a recognized regular-array wrapper), otherwise B. The label also fixes the task's target flow (A→pipeline, B→direct). Oracle labels use golden RTL that no generation-time component may see; they exist so that routing accuracy can be scored, and they define the ceiling arm (oracle) in the ablation of Section 6.7.

4.3 The Two-Tier Cascade

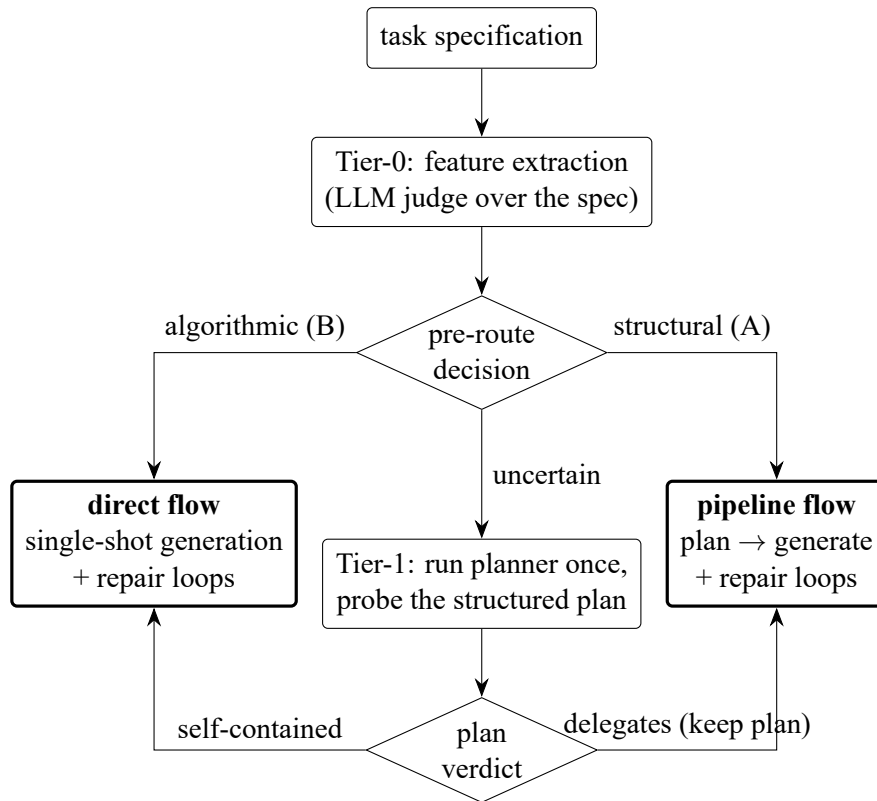
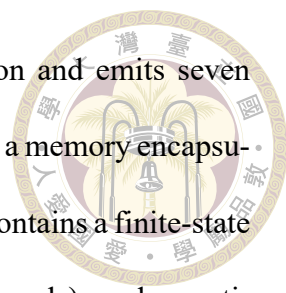


Figure 4.1: The two-tier cascade router. Tier-0 routes confident tasks from spec-level features alone; only uncertain tasks pay for a planner call, whose structured plan Tier-1 probes. A task routed to direct after planning discards the plan (a “wasted plan”).

The router is a cascade of two increasingly expensive tests (Figure 4.1); a task exits at the first tier that is confident.



Tier-0: spec pre-route. A feature extractor reads the specification and emits seven structured judgments: whether the module delegates to submodules, is a memory encapsulation, or is thin pass-through control (all class-A signals); whether it contains a finite-state machine, clock-domain crossing, or a datapath algorithm (class-B signals); and an estimate of the state bits the module itself must hold, plus a confidence value. The production extractor is a small LLM call that returns this JSON verdict (a regex-based keyword extractor over the same features serves as the ablation baseline — Section 6.7); the decision rule is deliberately transparent: any algorithmic signal routes to direct, else any structural signal routes to pipeline, else — or when the extractor’s confidence is below threshold — the task falls through as uncertain. One subtlety the LLM extractor handles and keywords cannot: a module can sound algorithmic (“preliminary decoding”) while actually being a wrapper that instantiates the real decoder, and only whole-spec reading resolves that.

Tier-1: plan probe. Uncertain tasks run the planner once — the cheapest informative pipeline component — and the router probes the resulting structured plan’s own natural-language requirement summary. Decisive delegation phrasing (“wraps”, “encapsulates”, “reuses existing”, or instantiation language without any own-implementation verb) keeps the task in the pipeline, plan in hand; clear self-contained-algorithm phrasing routes it to direct, discarding the plan. The asymmetry of the default is intentional: the plan is already a sunk cost, so the probe bails to direct only when the module is clearly self-contained. A plan generated and then discarded is counted as a wasted plan — the cascade’s overhead metric.

The direct flow. The direct arm is not a bare completion: it shares the generator’s repair machinery. A directly generated candidate that fails the internal lint enters the same

syntax repair loop, and (in arms where functional repair is enabled) the same functional repair loop, with the plan-dependent prompt sections simply absent. This matters for the routing economics: direct generation costs roughly 10–12 k tokens per sample against the pipeline’s 90–153 k, so every correctly-routed class-B task buys nearly an order of magnitude of compute reduction at equal or better quality.

4.4 Routing Metrics

Section 6.7 evaluates routing arms on four axes: coverage (solved tasks and pass@k, against the HYBRID union as the attainable ceiling); routing accuracy (the confusion between oracle class and chosen flow, where both misroute directions — A→direct and B→pipeline — forfeit a would-be pass); overhead (wasted plans); and efficiency (estimated tokens and summed compute per solved task). The design goal is stated once and measured throughout: reach the union coverage of both flows at the lowest compute, with misroutes near zero.





Chapter 5 Experimental Setup

This chapter fixes the experimental ground rules shared by every result in Chapter 6: the benchmark and its scoring semantics (Section 5.1), the models and serving infrastructure (Section 5.2), the metrics and the cost-accounting rules we commit to (Section 5.3), and a complete inventory of the experimental runs together with their configurations (Section 5.4). The inventory table doubles as the reproducibility anchor of this thesis: every number reported later traces back to exactly one row of Table 5.2.

5.1 The RealBench Benchmark

All end-to-end evaluations use RealBench [?], a Verilog generation benchmark built from three real, open-source IP projects rather than from isolated textbook exercises. Each task provides a natural-language specification of one target module, the source files of the modules it may depend on, a golden reference implementation, and a self-checking testbench. The generator must produce the target module; the surrounding project files are supplied by the evaluation environment.

The benchmark distinguishes module-level tasks, which target one module inside an otherwise-provided project, from system-level tasks, which ask for an entire subsystem. This thesis reports on the **60 module-level tasks** summarized in Table 5.1; system-level

Table 5.1: The 60 module-level RealBench tasks used in this thesis, by source project.

Family	Source project	Tasks	Character
AES	OpenCores AES-128 cipher core	6	self-contained datapath logic
SDC	OpenCores SD-card controller	14	bus-facing control logic
E203	Nuclei HummingBirdv2 E203 RISC-V core	40	deep module hierarchy, reuse-rich

tasks are excluded from all headline numbers because their specifications (up to ~ 704 k characters for `e203_cpu_top`) exceed what any of our serving configurations can process without aggressive condensation, and because a 60-task universe with three distinct families is already large enough to expose the phenomena this thesis studies. The three families span a useful difficulty gradient: AES modules are small and algorithmically self-contained, SDC modules implement Wishbone-facing control logic with wide interfaces, and the E203 family covers a complete RISC-V processor core whose modules instantiate one another — the setting in which IP reuse has the most to offer.

Scoring. RealBench scores with its own per-task Makefile, and its verdict is deliberately strict. A sample is counted as a syntax pass only if Verilator [?] compiles the full project — generated module, provided dependencies, reference model, and testbench together — with **no %Error and no %Warning** outside the Makefile’s explicit waiver list; warnings are fatal. A sample is a functional pass (“pass” for short) only if it also compiles and the testbench simulation reports zero output mismatches against the golden reference. Two consequences of this design recur throughout the thesis. First, our syntax numbers are conservative relative to benchmarks that fail only on errors (Section ??). Second, the environment imposes two non-negotiable structural contracts on generated code: a dependency that the environment ships as a file must be instantiated, never re-declared (re-declaration is a fatal MODDUP), and the testbench’s DUT instantiation fixes the exact port list (a missing port is a fatal PINNOTFOUND). Section 3.3 describes how the pipeline

internalizes both contracts.



5.2 Models and Serving Infrastructure

All generation uses open-weight models served locally through vLLM [?] behind an OpenAI-compatible endpoint, on university/NCHC GPU nodes reached through SSH port forwarding. Three models appear in this thesis:

- **gpt-oss-20B** [?] — the workhorse model. All ablations (grounding, repair cache, repair budgets, routing) run on this model, so their comparisons are model-controlled.
- **gpt-oss-120B** [?] — used to test which findings survive a $6\times$ scale increase (Sections 6.8 and 6.7).
- **Qwen3-235B** [?] — used in supplementary repair-budget runs; it does not participate in headline comparisons.

Two serving quirks shaped the implementation and are documented here because they are invisible in the results yet load-bearing for reproduction. First, the deployment returns answers in the `reasoning_content` field with an empty `content` field whenever tool schemas are attached to a request; the client transparently falls back to the `reasoning` field and, for the planner, re-prompts without tools (Section 3.2). Second, the server caps completion length rather than rejecting a request when prompt and requested tokens together exceed the 131,072-token context window, but returns HTTP 400 when the prompt alone exceeds it — the failure mode behind the “not generated” bucket of Section 6.8.

5.3 Metrics and Cost Accounting



Quality. For a run with n samples per task, we report the syntax rate and pass rate as per-sample means over all generated records, and pass@k per task with the standard unbiased estimator [?]

$$\text{pass}@k = \mathbb{E}_{\text{tasks}} \left[1 - \binom{n-c}{k} / \binom{n}{k} \right],$$

where c is the number of passing samples for the task, averaged over the 60 tasks. We additionally report solved tasks: the number of tasks with at least one passing sample, which equals $60 \cdot \text{pass}@n$ and is the quantity a practitioner who can afford n attempts actually cares about. Pass@k carries an independence assumption across a task’s samples; one of our own techniques (the run-scoped repair cache, Section 6.4) violates it by design, so all reported pass@k values come from arms where samples are independent, and run-scoped numbers are shown only to quantify that specific effect.

Cost. We use two cost measures. Estimated tokens are computed from prompt/response character counts with a calibrated 4.38-characters-per-token heuristic and summed per run; compute time (Σ_{wall}) is the sum of per-sample wall-clock seconds, which measures total machine work independently of the harness’s ~ 80 -way request concurrency. Three accounting rules apply throughout, each forced by a measurement hazard we identified in our own data: (i) token totals from runs that were resumed after interruption are not cited as costs, because resumed samples are re-read from disk with zero logged tokens; (ii) token totals for the direct flow are lower bounds, because its repair-loop calls are not instrumented; and (iii) wall-clock is never compared across runs executed on different days, because the shared vLLM server’s load varied — only within-run sums and per-

solved-task ratios are reported. These hazards and their blast radius are discussed together in Section ??.



5.4 Run Inventory

Table 5.2 lists every experimental run this thesis reports, the configuration that distinguishes it, and the section that consumes it. Short labels (left column) are used throughout Chapter 6. All runs share the same task discovery, per-task catalog construction, and RealBench evaluation harness, so rows differ only in the flow and the flagged configuration. Unless stated otherwise, sampled runs use temperature 0.2 (the runner default) and the two-stage pipeline of Chapter 3; “6/4” style entries abbreviate the syntax/functional repair-attempt budgets of Section 3.4.

Two rows deserve advance warning because they differ from their comparison partners in more than one variable. FUNC-10/10 changed both the repair budgets (2/4 → 10/10) and a scoring waiver (TIMESCALEMOD warnings suppressed) relative to FUNC-2/4; Section 6.6 disentangles the two. And HYBRID predates the direct-flow repair loop that the router runs include, besides using the larger 10/10 budgets; Section 6.7 therefore treats ROUTER-T1 → ROUTER-2T as the only single-variable routing comparison and labels the comparison against HYBRID as directional. One benchmark-side event also splits the timeline: the RealBench AES S-box testbench was repaired upstream on June 25, 2026 after we reported the race documented in Section 6.9; runs before and after that date are never compared on the two affected tasks.

Table 5.2: Inventory of all reported runs. Every number in Chapter 6 traces to one row. Directory names are given in Table 5.3.

Label	Model	$60 \times n$	Temp.	Distinguishing configuration
<u>Baselines and headline pipeline</u>				
DIRECT-T0	20B	$\times 1$	0	spec $\rightarrow$ model, no plan, no repair
PIPE-T0	20B	$\times 1$	0	grounded planner + generator, cache off
<u>Repair-cache trio (identical except cache scope)</u>				
CACHE-OFF	20B	$\times 20$	0.2	repair cache off
CACHE-TASK	20B	$\times 20$	0.2	repair cache, task scope
CACHE-RUN	20B	$\times 20$	0.2	repair cache, run scope
<u>Model scale</u>				
PIPE-120B	120B	$\times 10$	0.2	as CACHE-RUN, tool-calling probe
<u>Functional repair and budgets</u>				
FUNC-2/4	20B	$\times 10$	0.2	functional repair + spec slice, budgets 2/4
FUNC-10/10	20B	$\times 10$	0.2	as above, budgets raised to 10/10
<u>Routing (Chapter 4 arms)</u>				
HYBRID	20B	$\times (10+10)$	0.2	no router: both flows on every task
ROUTER-T1	20B	$\times 10$	0.2	plan-probe tier only, budgets 6/4
ROUTER-2T	20B	$\times 10$	0.2	full cascade (Tier-0 LLM + probe), 6/4
ROUTER-2T-120B	120B	$\times 10$	0.2	as ROUTER-2T

Table 5.3: Mapping from run labels to repository directories under runs/.

Label	Directory
DIRECT-T0	realbench_direct_model
PIPE-T0	agentic_plan_legacy_realbench_plan_hallu_fix_t0
CACHE-OFF	agentic_plan_legacy_realbench_oss20b_plan_hallu_no_rep_cache
CACHE-TASK	agentic_plan_legacy_realbench_oss20b_plan_hallu_task_rep_cache
CACHE-RUN	agentic_plan_legacy_realbench_oss20b_plan_hallu
PIPE-120B	agentic_plan_legacy_realbench_oss120b_plan_hallu_tool_call
FUNC-2/4	repair_spec_slice_oss20b_func_v2
FUNC-10/10	repair_spec_slice_oss20b_func_v2_time-err-off_10syn_10func
HYBRID	hybrid_oss20b_10syn_10func_rep_spec
ROUTER-T1	T1_router_oss20B_sync6_func4
ROUTER-2T	Full-2T_router_oss20B_sync6_func4
ROUTER-2T-120B	Full-2T_router_oss120B_sync6_func4



Chapter 6 Results and Analysis

This chapter evaluates the pipeline (Sections 6.1 through 6.6), the router (Section 6.7), and then turns the instruments on the failures themselves (Sections 6.8 and 6.9). All runs are identified by the labels of Table 5.2; all rates were recomputed directly from each run's per-sample records rather than transcribed from run logs.

6.1 Does Planning and Reuse Beat the Model Alone?

The cleanest head-to-head is deterministic: one greedy sample per task, identical harness, PIPE-T0 versus DIRECT-T0 (Table 6.1, Figure 6.1).

Table 6.1: Grounded pipeline vs. pure model on the 60 module tasks, temperature 0, one sample per task.

Arm	Syntax	Pass
PIPE-T0 (grounded planner + generator)	37/60 = 61.7 %	15/60 = 25.0 %
DIRECT-T0 (pure model)	16/60 = 26.7 %	6/60 = 10.0 %

The pipeline delivers $2.3\times$ the syntax rate and $2.5\times$ the pass rate of the same model prompted directly. The family breakdown locates the entire effect. On AES — small, self-contained, nothing to reuse — the model alone nearly saturates syntax (83 %) and the pipeline merely does not hurt (pass unchanged at 33 %). On SDC the pipeline helps modestly (pass 21.4 % $\rightarrow$ 28.6 %). On E203, the family whose modules instantiate one

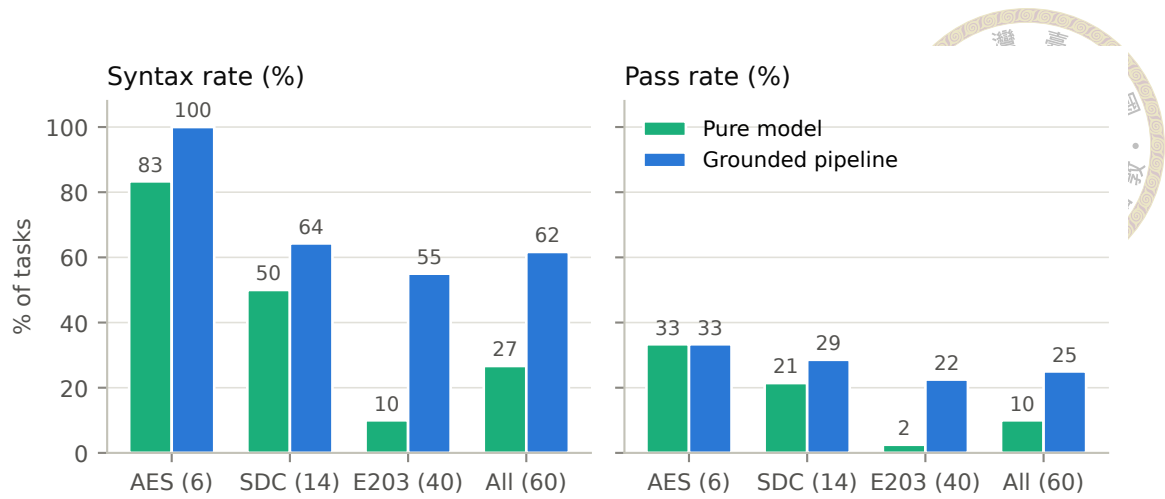


Figure 6.1: Syntax and pass rates by family, pure model vs. grounded pipeline (temperature 0, one sample). The aggregate gain is carried almost entirely by the reuse-rich E203 family.

another, the pipeline is decisive: syntax 10.0 % $\rightarrow$ 55.0 %, pass 2.5 % $\rightarrow$ 22.5 %. The conclusion we draw — and will lean on again when routing — is that the pipeline’s value is proportional to how much legitimate IP reuse a task affords. Where there is nothing to ground, planning is overhead; where the task is an exercise in wiring real submodules, injecting their true interfaces roughly decouples the syntax rate.

6.2 The Grounding Audit, and an Honest Null Result

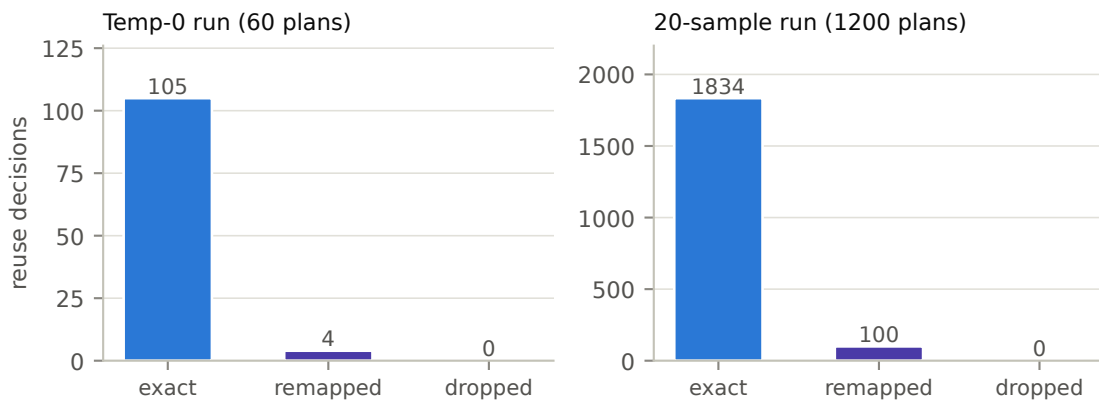


Figure 6.2: Grounding outcomes for every reuse decision in two runs. Near-miss names are remapped onto real catalog ids; nothing hallucinated survives into the plan.

Grounding-by-construction (Section 3.2) does exactly what it was built to do. In

PIPE-T0, the 60 plans contain 109 reuse decisions: 105 already name an exact catalog id, 4 near misses are remapped, and 0 hallucinations survive (Figure 6.2). At twenty samples (CACHE-RUN), the same mechanisms process 1,934 decisions: 1,834 exact, 100 remapped, 0 leaked. Before the fix, plans routinely carried invented module names; after it, every reuse decision in every plan resolves to a real, provided module — and the planner’s reuse matrix, previously empty because decisions hid under a dozen ad-hoc JSON keys, became fully populated and auditable.

And yet the end-to-end effect on quality is nearly zero: pass@1 moved from 19.6 % (ungrounded twin run) to 20.2 % — +0.6 percentage points, within noise. We report this null result deliberately. The generator was already silently discarding invented names and re-deriving reuse from the real dependency set, so grounding purchased plan correctness and auditability, not yield. Plan quality was not the binding constraint; Sections 6.8 and 6.9 show what is.

6.3 Sampling Structure: pass@1 to pass@20

Figure 6.3 shows the sampling structure of the pipeline on CACHE-OFF. Three facts matter downstream. First, diversity helps but saturates fast: pass@1 is 18.5 %, pass@5 already 26.6 %, pass@20 only 31.7 % — samples 6–20 add less than the first four. Second, the task universe splits sharply: 19 of 60 tasks are solved at least once in twenty tries; the other 41 never pass, at any k . Third, greedy decoding beats the sampled mean: the deterministic PIPE-T0 scores 25.0 % against the temperature-sampled per-sample mean of 18.5–20.2 % — the standard precision-versus-diversity trade (temperature is what buys the pass@20 headroom), with the caveat that the temp-0 and sampled runs also differ in

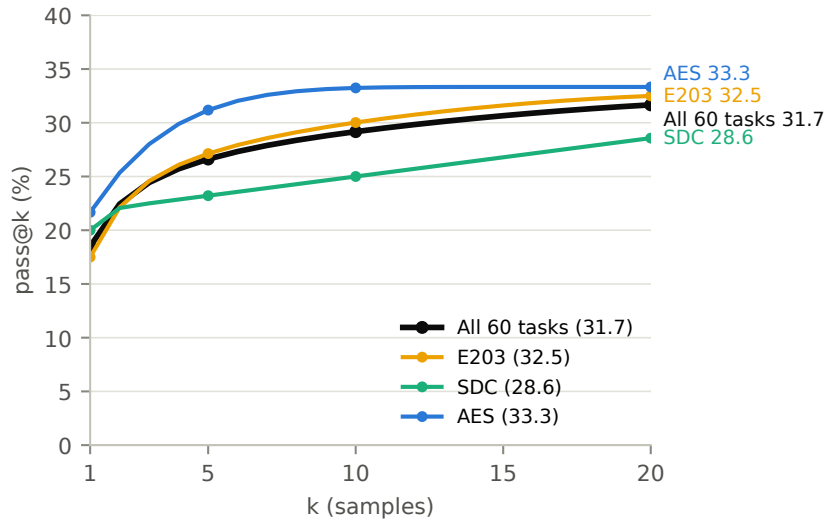


Figure 6.3: Unbiased pass@k on CACHE-OFF (60×20). Diversity buys +13 points from $k=1$ to $k=20$, but most of it is realized by $k=5$.

search mode, so the comparison is directional.

6.4 Repair-Cache Ablation: Density, Not Solvability

The cache trio (CACHE-OFF/-TASK/-RUN) differs in exactly one flag, giving the thesis’s cleanest three-arm ablation (Table 6.2, Figure 6.4).

Table 6.2: Repair-cache scope ablation (60×20 , gpt-oss-20B, identical configuration otherwise).

Cache scope	Syntax	pass@1	pass@20	mean repair attempts
off	59.4 %	18.5 %	31.7 %	1.12
task	64.3 %	19.6 %	31.7 %	1.32
run	66.9 %	20.2 %	31.7 %	1.61

The cache does what a syntax-hint mechanism can do: verified fix hints lift the syntax rate monotonically (+4.9 pp at task scope, +7.5 pp at run scope) and pull per-sample pass up with it, at the cost of more repair turns taken. What it cannot do is change which tasks are solvable: pass@20 is identical to the decimal across all three scopes. The cache shifts the density of passing samples inside already-solvable tasks; functional bugs — the actual

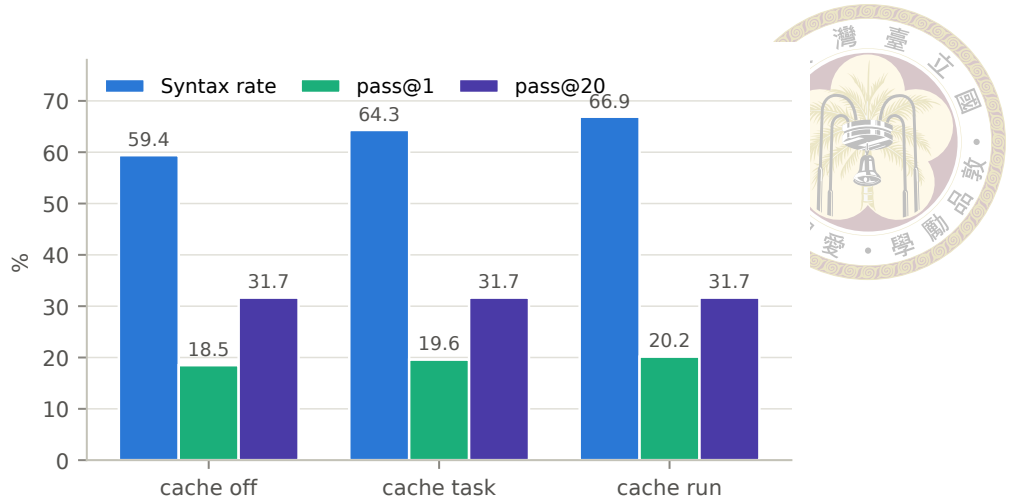


Figure 6.4: The cache lifts syntax monotonically and drags pass@1 with it, but pass@20 does not move at all.

frontier — are immune to syntax-diff hints.

Two qualifications. First, these are three independently sampled runs, so the monotone ordering of syntax and pass together is suggestive rather than proof; individual deltas of 1–2 pp are within sampling noise. Second, run scope shares hints across a task’s twenty samples, violating pass@k’s independence assumption — run-scope pass@k is shown only to demonstrate that even with that inflation pressure the solvable set does not budge.

Task scope is the methodologically safe default, and it is what later runs adopt. Token totals across this trio are not comparable (the task/run arms were resumed mid-run; Section ??); the one clean cost figure is CACHE-OFF’s 46.3 M estimated tokens for 1,200 samples.

6.5 Functional Repair: a Separate, Oracle-Fed Arm

The functional repair loop and specification slicing (Sections 3.4.2–3.4.3) are evaluated as their own arm because their feedback signal is the scoring testbench itself. Raising the budgets from 2/4 (FUNC-2/4) to 10/10 (FUNC-10/10) lifts syntax from 60.3 % to

81.5 % and pass from 20.2 % to 27.5 % — but at $2.7\times$ the cost (33.5 M $\rightarrow$ 89.8 M tokens). Attributing the +44 passing samples is sobering: +37 of them are candidates that, thanks to deeper syntax repair, now compile and then pass the testbench on their first functional check; only +7 net passes are attributable to the functional repair loop itself. (The two arms also differ in one scoring waiver — TIMESCALEMOD warnings suppressed in FUNC-10/10 — so the syntax delta slightly overstates the budget effect alone.) The loop's value proposition — oracle feedback included — is thin; the next section shows why.

6.6 Repair-Budget Calibration: Fix Fast or Never

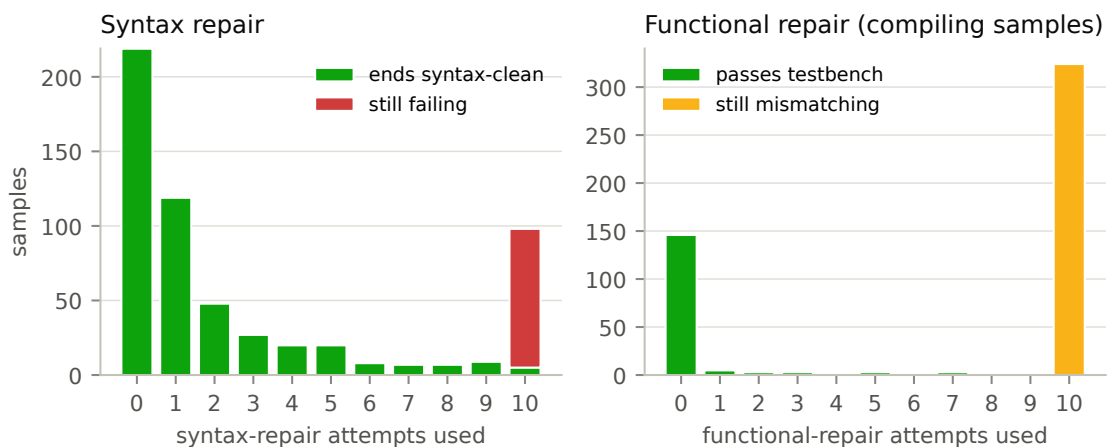


Figure 6.5: FUNC-10/10: outcomes by repair attempts consumed. Left: samples by syntax-repair attempts; nearly every bucket short of the cap ends syntax-clean, while the cap bucket (10) is 95 % failures. Right: compiling samples by functional-repair attempts; every rescue lands by attempt 7, and 324 samples burn the full budget converting nothing.

Both repair phases share one shape — fix fast or never — but with very different economics (Figure 6.5).

Syntax repair earns its budget. In FUNC-10/10, 219 samples compile without repair; repair rescues another 265 by attempt 9, with meaningful yield still arriving at attempts 4–6; the cap bucket is where hope goes to die (98 samples reach attempt 10; 5 succeed).

The independent ROUTER-2T run confirms the shape at budget 6/4: 202 born-clean, 234 repair-rescued with a real tail across all six attempts (77/54/44/23/22/14), 164 exhausted. A syntax budget of about six captures nearly all attainable value.



Functional repair does not. Of the compiling samples in FUNC-10/10 that enter the functional phase failing, the loop rescues 19 — every one by attempt 7 — while **324 samples burn all ten attempts and convert zero**. Since each attempt is a full compile-plus-simulation (up to 300 s) plus an LLM call, 3,240 of 3,324 functional-repair attempts — 98 % of the phase’s compute — produced no pass. This is where the arm’s $2.7\times$ cost blow-up lives. ROUTER-2T again confirms at scale: 13 functional rescues against 252 exhausted samples, and on its direct flow the loop is essentially dead (2 rescues in 342 samples). The prescription adopted for later runs: syntax cap ≈ 6 , functional cap ≤ 4 , plus a no-progress early exit (abort when the mismatch count stops dropping). The deeper reason functional repair fails so often — the mismatch signal itself is frequently uninformative or misleading — surfaces in Section 6.9.

6.7 Routing: Union Coverage at a Fraction of the Compute

Table 6.3: Routing arms (gpt-oss-20B). Solved = tasks with ≥ 1 passing sample. Σ wall sums per-sample wall-clock. Token totals marked + are lower bounds (direct-flow repair calls are not instrumented). HYBRID and FUNC-10/10 use budgets 10/10 and predate direct-flow repair; the single-variable comparison is ROUTER-T1 $\rightarrow$ ROUTER-2T.

Arm	Samples	pass@10	Solved/60	Σ wall (h)	Mtok/solved
HYBRID (both flows)	600+600	46.3 %	30	423	3.24 ⁺
FUNC-10/10 (pipeline only)	600	38.3 %	23	402	3.90
ROUTER-T1 (probe only)	600	35.0 %	21	213	2.17
ROUTER-2T (full cascade)	600	56.7 %	34	130	0.83⁺

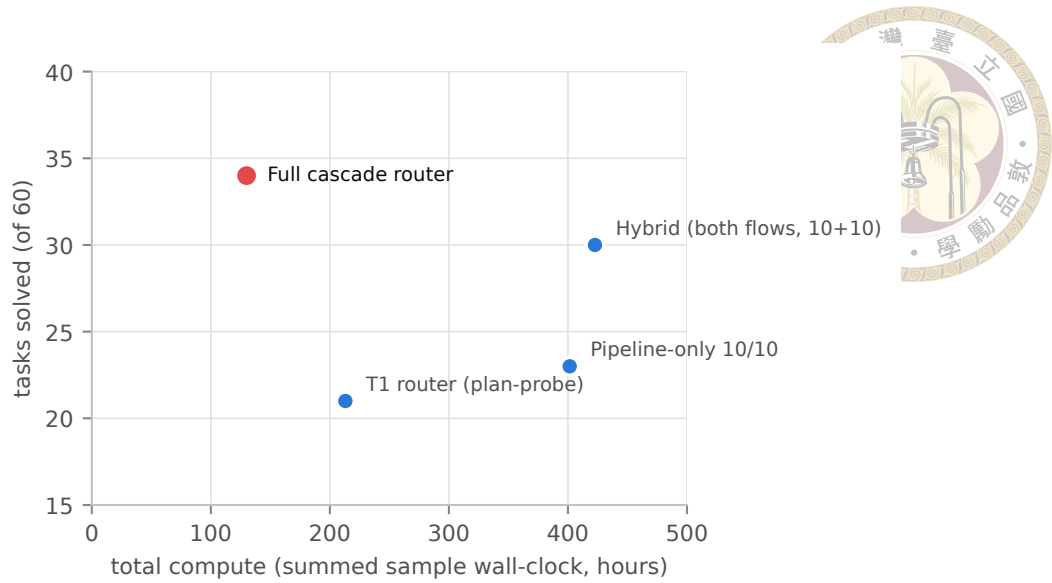


Figure 6.6: Tasks solved vs. total compute. The full cascade router solves the most tasks of any arm at less than a third of the compute of running both flows everywhere.

Table 6.3 and Figure 6.6 give the routing result. Three comparisons carry the argument.

Tier-1 alone is toothless. ROUTER-T1 exposes only the plan probe, and the probe — which by design defaults to keeping the sunk-cost plan — routed every task to the pipeline. The arm therefore collapses to pipeline-only at budget $6/4$ (21 solved), and usefully so: it is the controlled base for the cascade.

The cascade is a strict win over its own ablation. ROUTER-T1→ROUTER-2T changes exactly one thing: Tier-0 LLM pre-routing in front of the probe (which then handles only the 5 of 60 tasks Tier-0 deferred). Solved tasks jump $21 \rightarrow 34$ and $\text{pass@10 } 35.0\% \rightarrow 56.7\%$ while total compute falls $213 \rightarrow 130$ hours — more coverage for less work, because 342 of 600 samples run on the direct flow at roughly a tenth of a pipeline sample’s token cost. Per solved task, the cascade spends 0.83 M tokens against the brute-force hybrid’s 3.24 M — roughly $4\times$ more token-efficient.

Against the hybrid ceiling, directionally ahead. ROUTER-2T solves 34 tasks where HYBRID — running both flows on everything — solves 30 at $3.3\times$ the compute. This comparison is not single-variable: the hybrid predates the direct-flow repair loop (its direct samples compiled at 24.2 % vs. the router run’s 76.0 %) and used 10/10 budgets, so part of the router run’s edge is repair, not routing. We therefore claim only that the cascade reaches at least union-level coverage at a fraction of the cost, not that it strictly beats the union. Notably, the two hybrid-solved tasks the router run misses (e203_exu_branchslv, e203_ifu_ifl2icb) were correctly routed — both are direct-only tasks that the hybrid’s pipeline flow also never solved (0/10 each) — and were lost to sampling variance on low-yield tasks (2/10 and 1/10 direct yields), not to the routing policy or the trimmed budgets.

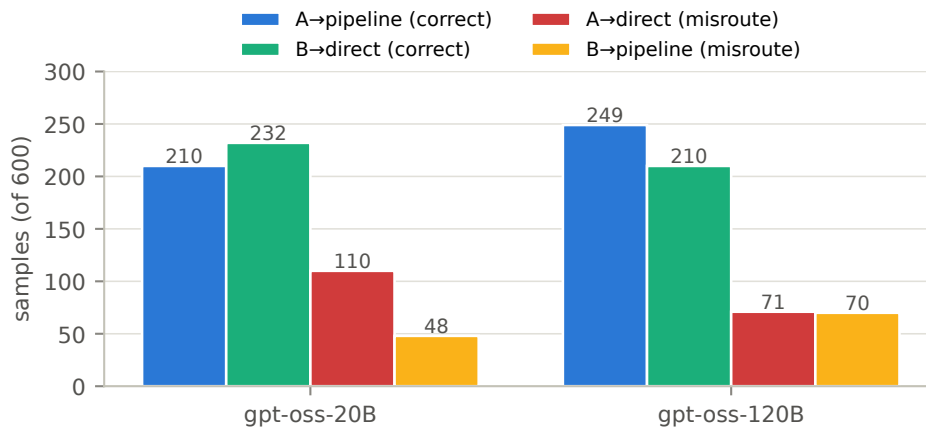


Figure 6.7: Flow allocation against oracle task classes for the full cascade, both model sizes. Misroutes concentrate in A→direct: pipeline-favoring tasks sent to the cheap flow.

Where the router is still wrong. Scored against the oracle labels (Figure 6.7), the 20B cascade misroutes 158 of 600 samples (26 %), dominated by A→direct (110): wrapper tasks sent to the flow that cannot see their submodule interfaces. The probe tier is underused (5 of 60 tasks), and 22 of 279 generated plans are discarded — the cascade’s modest overhead. With gpt-oss-120B the same router solves the same 34 tasks at pass@1 41.2 % (vs. 30.7 %) and misroutes drop to 141, differently distributed (A→direct falls to

71; B→pipeline rises to 70): the bigger model’s feature extractor defers less and trusts the pipeline more. Closing the A→direct gap — e.g. by letting spec-level uncertainty trigger the probe more aggressively — is the router’s clearest headroom.



6.8 Failure Taxonomy: Three Tiers of Difficulty

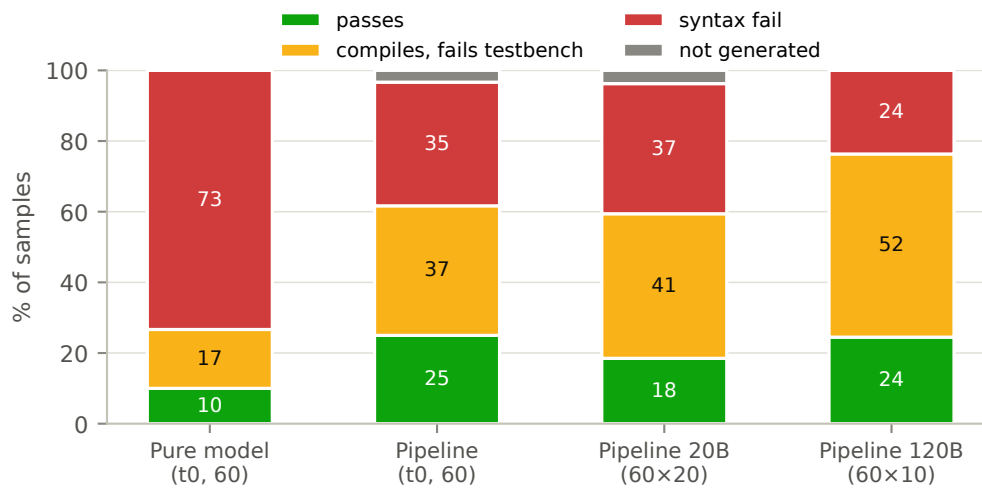


Figure 6.8: Per-sample outcomes. As models and repair improve, the red syntax band shrinks — and the amber compiles-but-fails-testbench band absorbs it. The functional share of compiling samples is essentially model-invariant.

Figure 6.8 decomposes every sample into four outcomes. The picture is consistent across runs: the largest failure bucket is not syntax but compiles-and-fails-the-testbench — right ports, right structure, wrong logic. Among compiling samples, that share is 59.5 % in PIPE-t0, 68.9 % in CACHE-OFF, and 67.9 % in PIPE-120B. The 6× larger model is a strictly better Verilog syntactician — syntax 59.4 % → 76.3 %, first-pass compiles up, repair turns down — but it is not a better logician: the functional share of compiling samples does not move. Scale fixes syntax, not semantics; this is the strongest evidence that the functional ceiling is structural rather than a capability artifact of the 20B model.

Reading failures per module (Figures 6.9 and 6.10) resolves the aggregate into three

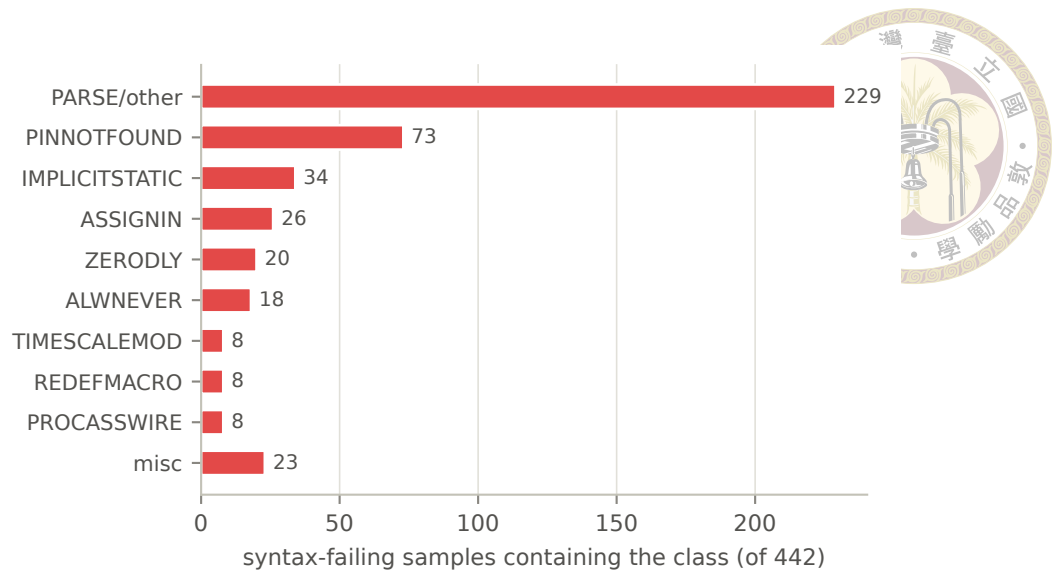


Figure 6.9: Verilator diagnostic classes among the 442 syntax-failing samples of CACHE-OFF (a sample can carry several classes).

stable tiers:

1. **Solved: leaf, RAM, and glue modules.** `aes_key_expand_128`, the CRC and RAM blocks, clock gating, register files — small, well-specified, often pure reuse. The pipeline has essentially retired this tier.
2. **Logic-limited: datapath and control modules.** Most AES round logic and the E203 ALU/decode/commit/dispatch cluster compile with perfect syntax and fail the testbench — some at 0 passes despite dozens of structurally distinct compiling candidates. The syntax-only repair loop is blind here by construction.
3. **Scale-limited: giant integrators.** `e203_core/cpu/exu/lsu`, `sdc_controller`: hundreds of ports, dozens of submodules. Failures are interface-completeness (PINNOTFOUND concentrates exactly here), context overflow (the not-generated bucket: timeouts, empty returns, and HTTP 400s on the biggest specs), and integration logic. Notably, plan-level interface hallucination reappears only on this tier — the `e203_exu` plan promotes 25 internal signals to phantom top ports while dropping 10 real ones —

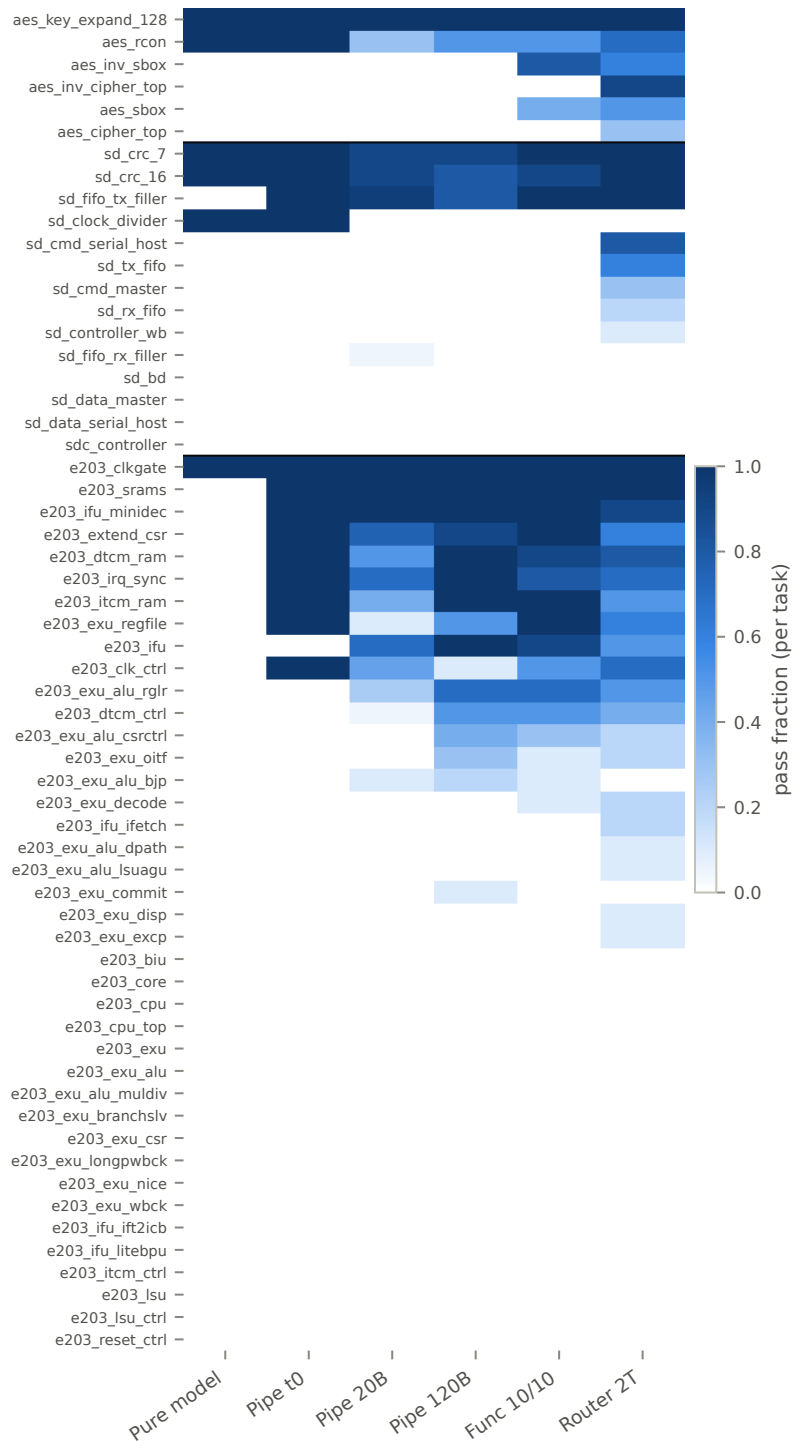


Figure 6.10: Pass fraction per task (rows, grouped AES/SDC/E203) across six runs (columns). The three difficulty tiers are visible as texture: saturated rows (leaf/glue modules), intermittent rows (logic-limited datapath modules), and blank rows (giant integrators).

the same interface-too-large-to-hold phenomenon at a different stage.

Each tier wants a different intervention — nothing (tier 1), functional feedback (tier 2), decomposition and contract enforcement (tier 3) — which is why aggregate pass rate alone is a blunt instrument for steering this system.

6.9 When the Benchmark Is Wrong: Scoring Artifacts

Treating every 0% module as model failure would misdirect the next round of engineering, so the persistent zeros were audited individually with a golden-model methodology: run the benchmark's own reference implementation through the full scoring path, and if the golden fails (or a byte-equivalent candidate fails), the task's zeros are artifacts, not model errors. Three audits produced three different verdicts.

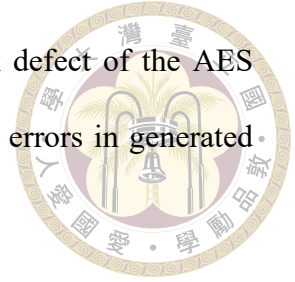
The AES S-box race (confirmed artifact, since fixed upstream). `aes_sbox` and `aes_inv_sbox` scored 0 across all runs with one suspicious signature: every sample — combinational, ROM-array, or registered variants alike — reported exactly 766 mismatches. Diffing showed generated lookup tables byte-identical to the golden. The cause is a testbench race: stimulus changes the input and samples the output in the same simulation timestep, so any correct-but-differently-scheduled implementation (e.g. a continuous assign where the reference uses blocking always @(a)) settles one delta cycle late at each of the ~ 768 input transitions. The trigger was isolated by incremental flag bisection to Verilator's `--coverage-line` instrumentation — the identical design passes without it — and the recorded mismatch count decomposes exactly as $766 + 3 \times (\text{wrong table bytes})$, meaning the artifact also corrupts mismatch counts as a repair signal: a one-byte-wrong design

(0.4 % wrong) reports ~ 50 % mismatches. A registered-output module using the same testbench template (`aes_key_expand_128`) passes 20/20, completing the case that this is a combinational-settling race. Impact: 21 byte-perfect samples scored zero in CACHE-OFF alone, and 2 of 60 tasks were unsolvable purely by artifact. The finding was reported and the benchmark's testbench was repaired upstream on June 25, 2026 (settle before compare); pre-/post-fix runs are never compared on these two tasks.

The SDC controller (audited, genuine). `sdc_controller` — 0 functional passes in 220 records — looked like a candidate for the same treatment and is the reason the golden methodology matters: its golden implementation passes its testbench perfectly (0 mismatches in 23,932 checks) under the full scoring flags, because all its outputs are registered and immune to the settling race. The zeros are honest: 87 % of SDC samples die on syntax through interface hallucination introduced at the generation stage (instantiating provided submodules with invented pins, inventing a different master-bus naming, dropping the `sd_defines` macros) even when the plan's interface is correct — and the 13 % that compile are grossly wrong (~ 95 % mismatch rates), not near misses.

The repair gate that fights the scorer (pipeline artifact). Auditing the 97 samples of FUNC-10/10 that exhausted all ten syntax repairs found that 42 of them (43 %) fail on a fatal warning located in a benchmark harness file (`stimulus_gen/ref`) — code the model cannot edit. The internal repair gate (`verilator --lint-only`) treats three warning classes as fatal that the actual scorer's build does not even emit, so on four tasks every sample burned its entire repair budget “fixing” correct code and never reached the functional phase. Golden test again: three of the four goldens pass the real scoring build (pure pipeline artifact; the fix is to mirror the scorer's waivers in the gate), while the fourth

(e203_reset_ctrl) fails even as golden — a genuine benchmark defect of the AES class. The remaining 53 % of repair-exhausted samples fail on real errors in generated code, concentrated on the giant integrators of tier 3.



The methodological point generalizes beyond this benchmark: in a harness this strict, persistent zeros deserve a golden-model audit before they are allowed to steer engineering effort — of our three audits, one exonerated the models, one convicted them, and one convicted our own tooling.





Chapter 7 Discussion, Limitations, and Conclusion





References





Appendix A — Introduction

A.1 Introduction

A.2 Further Introduction





Appendix B — Introduction

B.1 Introduction

B.2 Further Introduction